ChatGPT

面向 2026 的 Tool Use 研究方向重构：从表层 Function Calling 到潜在决策与后见校正

问题重构

你提出的核心分歧（“tool use 不是输出 JSON 的模式匹配，而是决策”）在 2026 年是成立的，而且越来越接近“共识但尚未被正确建模”的状态：一方面，主流 agentic RL/后训练工作已经把 **动作= token 序列** 明确写进 MDP 形式化，强调 **next-state signal**（用户回复、工具返回、报错、GUI 状态变化）同时包含 evaluative 与 directive 信息；另一方面，大量 tool-use 工程/训练仍停留在 **tool identity / schema string** 的表层拟合，导致泛化到“工具名改了/描述换了/参数组织变化了/同意图多策略都合理”的时候系统性脆弱。 [1]

更关键的是，你给出的统一视角
$s_t \to z_t \to y_t$，且 **next-state feedback 主要用来修** z_t，在 2025–2026 的文献里确实呈现出一种“碎片化覆盖”：

- 已有很多工作在 $s_t \to y_t$（直接生成/修 token）上越做越强：例如从 next-state 提取文本提示、把“反馈条件下的模型”当 teacher，再蒸馏回“无反馈”的 policy，从而将 rich feedback 变成密集的 token-level 学习信号。 [2]
- 也有工作在 $s_t \to z_t$ 上开始出现：以 CoLA 系列为代表，通过 **逆动力学 + codebook latent action** 把 action 空间从巨大 token vocab 压缩到小 codebook，用于提高 RL 探索效率或语义多样性。 [3]
- 但几乎没有工作在 **“用 next-state feedback 去显式更新 z_t 的后验/策略几何”** 上成体系：大多数还是把反馈编译成 token-level 梯度，或把 latent 当“更好探索/更省算力”的接口，而不是“决策语义的对象”。 [4]

这就解释了你为什么会自然想到两步“升级”：

1) **Latent action**：不是为了更会输出 function call 格式，而是把 tool-use 的“选择/顺序/抽象粒度/何时停止/何时补参数”变成 **可迁移的决策变量**，降低对表层 schema 的敏感性。CoLA/coverage-enhanced latent actions 明确动机就是：token 级 RL 的探索空间极大、难以有效探索，于是用 latent codebook 作为动作空间。 [5]

2) **Diffusion / flow**：当同一状态存在多条合理策略（multi-modal policy），单峰自回归/离散分类会“平均化或坍缩”。在连续控制里，flow/diffusion 近两年被反复强调的优势正是 **“能拟合复杂、多峰的动作分布”**；与此同时，语言侧也出现了把 diffusion 放在 latent planning 而不是 token surface 的路线（例如把生成过程切成“stop-think（latent diffusion plan）-autoregress（token realization）”）。 [6]

3) **Hindsight feedback**：next-state 自带的 directive 信息（报错栈、工具返回、用户纠正）本质上是在告诉你“上一时刻的决策应该如何不同”，如果只在 token 空间蒸馏，很容易得到“更会把错因写对/更会补格式”，但未必学到“下一次换个 schema/换个工具也应该做同类决策”。OpenClaw-RL 的核心叙事就明确把 next-state signal 分成 evaluative（可做 PRM/标量 reward）与 directive（可做 token-level directional advantage/OPD）。你要做的，是把 directive 的箭头对准 z_t。 [2]

2026 年是否成立?
成立；而且“只做更强 schema encoding/更强工具描述匹配”在 2026 的 novelty 预期会明显变差：一方面，Hammer 类工作已经直接指出 function-calling 会过拟合命名约定，并用 masking 等方式提高鲁棒性；另一方

面，ToolFlood 这类工作从对抗角度说明 embedding-retrieval 这一层本身就可能被几何结构击穿。两者都在强调：决定成败的不是“能不能输出格式”，而是“决策对象到底是什么”。 [7]

文献图谱

latent action 与 abstract policy

CoLA（ICML 2025）
解决什么：把 LLM 的 token-level action space 改造成 **compact latent action space**，以提高 RL 的可控性与探索效率。 [8]
核心方法：逆动力学从“未来 token/未来观测”反推出当前 latent action；同时把预训练 LLM 变成能接收 latent action 的“language world model”，再训练 policy 产生 latent action（可 BC 或 RL）。 [8]
与你最相关：它把“动作”从表层 token 序列抽象成 z_t，且明确在做 agent for RL 的结构化。 [8]
没解决什么：z_t 的语义稳定性/可解释性并非目标函数直接保证；更重要的是，它把“反馈”仍然当作 RL reward 或任务分数，并没有把 **next-state directive** 显式对齐到 z_t。 [9]

Coverage-Enhanced Latent Actions（2026，多模态对话 agent）
解决什么：把 latent action RL 扩展到多模态对话 agent，并指出 codebook 覆盖不足是瓶颈，于是用 paired + text-only 数据、跨模态 projector、cycle consistency 来扩大 latent codebook 覆盖。 [10]
核心方法：明确把动作空间从 token vocab（例：152K）压到 latent codebook（例：128）；policy 输出离散 latent action index，world model 仍 auto-regress 生成 token。 [10]
与你最相关：它把“latency/探索空间”问题讲得非常直白，也提供了一个现实提醒：**latent action 的主要收益可能只是“更好探索/更少算力”，而不是“更语义的决策”**。 [10]
没解决什么：它解决的是 codebook coverage，而不是 “schema-invariant decision” 或 “hindsight 修 z”；latent 仍是离散 codebook，表达多峰策略的能力很可能受限于 codebook 粒度与训练数据的生成分布。 [10]

What Do Latent Action Models Actually Learn?（NeurIPS 2025）
解决什么：从理论角度质疑 LAM 是否真的学到“可控动作因素”，还是在压缩观测差分/噪声；给出线性化分析，指出 LAM 在一定条件下会退化为对“可控变化+外生噪声混合”的 PCA，并讨论数据生成策略随机性、augmentation、aux action prediction 等对 latent 语义的影响。 [11]
与你最相关：这是对你“latent 是语义决策还是压缩码”的直接证据链：**如果训练目标只是重构/预测差分，latent 可能不对齐决策语义**，除非你把“可控性/可执行性/后验一致性”写进 objective。 [11]
没解决什么：它是诊断与条件分析，不是面向 tool-use/next-state directive 的端到端方案；但它提供了你写 rebuttal/写 related-work gap 的强抓手：不先解决“latent 对齐对象”，后续在 latent 上做 diffusion 可能只是扩大一个不稳定表征的容量。 [11]

diffusion / flow decision models

FlowRL（NeurIPS 2025）
解决什么：强调 RL 性能不仅取决于训练信号，也取决于 policy class 的表达力；flow-based generative models 擅长刻画复杂、多峰动作分布，但直接用于 online RL 会有“静态 imitation vs 动态 value optimization”的目标不匹配，因此提出 Wasserstein-2 约束的 policy search 形式来对齐目标。 [12]
与你最相关：它给出了“为什么要用 flow 而不是离散分类/单峰 AR”在 RL 语境里的正式论证：**多峰策略 + policy class 表达力**。 [12]
没解决什么：主要在连续控制；如何把这种“多峰分布的几何优势”迁移到语言 agent 的 z_t（尤其是离散/半离散 latent）并不自动成立。 [13]

pi_RL（2025）
解决什么：讨论 diffusion policy 在 RL 中的核心难点之一：action log-likelihood 难算（迭代去噪导致），从

而影响策略梯度与训练稳定性；提出把 score-based diffusion policy 转成可 tractable 的形式（Flow-Noise / Flow-SDE）以便 RL。 [14]
与你最相关：这是你在论文里必须正面回答的“diffusion/flow 的训练稳定性与 credit assignment”证据：它不是“自然延伸就能跑”，而是需要解决 objective/likelihood/梯度估计的问题。 [15]
没解决什么：仍是连续动作；对语言 latent policy 是否需要显式 likelihood（还是只需采样+选优）要重新论证。 [16]

What Makes a Good Diffusion Planner for Decision Making?（ICLR 2025）
解决什么：系统性梳理 diffusion planning 在 offline RL 的关键设计，并声称训练评估了 6000+ diffusion models，识别 guided sampling、架构、action generation 与 planning strategy 等关键组件。 [17]
与你最相关：它把 diffusion 用在“高层 decision/trajectory”而非单步控制上，且强调 guided sampling 与 planning strategy 选择会决定性能——这与你想把 diffusion 放在 “latent plan / tool strategy” 的上层很对齐。 [17]
没解决什么：它回答的是 offline RL diffusion planner 的工程机制，不会自动给出“如何用 next-state directive 修 latent decision”的学习原则。 [18]

SAGE（2026）
解决什么：指出 diffusion planner 可能生成“值高但动力学不一致”的轨迹，导致执行脆弱；提出用 self-supervised latent consistency signal 定义 energy，对候选轨迹 re-rank（无需环境 rollout、无需重训 policy）。 [19]
与你最相关：它提供了一个很关键的替代视角：**diffusion 负责覆盖候选，energy/consistency 负责“可执行性”筛选**。把它搬到 tool-use，就是“生成多种工具策略，再用可执行性/一致性能量筛选”。 [20]
没解决什么：如何学习能量/一致性使其对齐真实 tool affordance，而非 dataset spurious correlations，需要在语言环境里重建。 [21]

Diffusion on Syntax Trees for Program Synthesis（ICLR 2025 Spotlight）
解决什么：针对 code/token 自回归缺乏“看到程序输出后的反馈”这一痛点，提出在语法树上做 diffusion，保证语法合法性，并便于结合 search，实现“写程序—执行—debug—再编辑”的迭代。 [22]
与你最相关：这是“diffusion 不只适用于连续控制，也能用于高层离散结构（AST/plan）”的强例子；而 tool strategy/plan 本质上更接近“结构对象”而非连续控制。 [23]
没解决什么：它的反馈闭环更像 search/debug，并非把 next-state feedback 作为 learning signal 显式写进 latent decision space（更多是 inference-time loop）。 [24]

STAR-LDM（Stop-Think-AutoRegress，2025/2026）
解决什么：提出把生成拆成“停下来在连续 latent 空间做 diffusion planning，再恢复 AR 生成 token”，强调这能在做出不可逆 token 决策前进行全局规划，并提升可控性与一致性。 [25]
与你最相关：这是你 $s \to z \to y$ 视角在语言生成/规划上的直接先例：**diffusion 放在 latent plan 而不是 surface token**，并明确强调“token 决策不可逆”。 [26]
没解决什么：其 latent plan 是 sentence embedding/soft prompt 级别，不是 tool-use 的 action abstraction；也没有把 tool execution 的 next-state directive 当作 posterior 证据来更新 z_t。 [27]

CADD（2025）
解决什么：指出离散 diffusion 用 [MASK] 会造成“信息真空”，提出离散 masking + 连续 latent diffusion 的耦合，使 masked 位置保留“噪声但有信息”的语义 hint，并显式提出可在采样时调节 mode-coverage vs mode-seeking。 [28]
与你最相关：它提供了一个直接可迁移的论点：你要的不是“永远覆盖所有模式”，你要的是**可控的多峰性**（高熵状态覆盖多策略，低熵状态收敛到精准调用）。 [29]
没解决什么：仍是生成建模框架；如何用环境反馈塑形这种 mode balance，需要你把 hindsight/PRM 信号接进来。 [30]

dLLM 在 agent/tool calling 的现实检验（2026）
- “Bitter Lesson”工作指出 diffusion LLM 作为 agent backbone 在长程规划与 tool-calling（严格 JSON schema）上系统性失败，并观察到它更适合做非因果模块（如 tool selection、memory summarization）而不是承担全部决策链。[31]
- 另一条线（DLLM Agent）在控制匹配框架下比较 diffusion vs AR backbone，报告 diffusion agent 在端到端速度、交互轮数上可能有优势，但也强调 naive diffusion policy 更容易出现 structured tool-call failure，需要更强的 tool-call 专项训练与注意力/掩码对齐。[32]

这两者合在一起给你的关键启发是：**把 diffusion 放在 y_t（surface tool call token）风险很高；把它放在 z_t（非符号、可全局修订的决策变量）才更“结构匹配”。**[33]

Latent-DARM（ICLR 2026 workshop）
解决什么：把离散 diffusion model 当 planner、AR model 当 executor，并提出 latent-space communication 接口，声称比 text-based interface 更有效且节省 token budget。[34]
与你最相关：这是少数直接把“planner/executor 异质模型协作”放在 **latent interface** 的工作之一，形式上非常贴近你想要的“决策层 vs realization 层”分离。[35]
没解决什么：其任务是 reasoning 协作，不是 tool-use；且“hindsight 信号如何形成后验并更新 latent policy”仍未触及。[36]

hindsight / OPD / correction

OpenClaw-RL（2026）
解决什么：把 next-state signal（用户回复、工具返回、错误等）视作统一在线学习信号，区分 evaluative（可转 process reward via PRM）与 directive（可用 OPD 得到 token-level directional advantage），并强调这些信号跨对话/终端/GUI/tool-call 等设置都是普适的。[37]
核心方法：从 next state 抽取 hint，构造 enhanced teacher context，让“同一个模型在有 hint 条件下”产生更好的 token 分布，再蒸馏回学生，形成 token-level 优势方向信号。[37]
与你最相关：这几乎是你要的“hindsight feedback 不是只有标量 success”，并且明确给出一种把 directive 变成梯度的方法。[37]
没解决什么：它把 action 明确建模为“response token 序列”，OPD 的对象也是 token 分布；**它修的是 y_t，不是 z_t**。[37]

SDPO（Reinforcement Learning via Self-Distillation，2026）
解决什么：指出 RLVR 只用标量 outcome reward 存在严重 credit assignment 瓶颈，而很多可验证环境提供 rich textual feedback（例如 runtime error、judge 解释）；提出把“反馈条件下的模型”作为 self-teacher，把其 next-token 分布蒸馏回 policy，从而把反馈转成密集学习信号，并报告在 tool use、编程等任务上提升 sample efficiency。[38]
与你最相关：它从“token-level 蒸馏”角度把 rich feedback 变成训练信号，与你的 hindsight 动机完全一致。[39]
没解决什么：同样没有 latent decision 表征，更没有“把反馈对齐到 latent 后验”的机制。[40]

PRIME（Process Reinforcement through Implicit Rewards，2025）
解决什么：聚焦于 dense process rewards 与在线 PRM 更新难题，提出只用 policy rollouts + outcome labels 通过 implicit process rewards 进行在线 PRM 更新，宣称降低开发成本与 reward hacking 风险，并在 math/coding 上验证。[41]
与你最相关：它说明“反馈不止终局标量”是趋势且确有收益；但它的奖励仍是“token-level process reward”。[42]
没解决什么：它没有回答 directive（应该怎么改）这一类后见信息的结构化建模，也没有 latent space。[42]

ReasonFlux-PRM（2025）
解决什么：指出以往 PRM 难以稳健评价长 CoT/trajectory 类型推理轨迹，于是做 trajectory-aware PRM，结合 step-level 与 trajectory-level supervision，并讨论其在数据选择、RL、test-time scaling 中的用途。[43]
与你最相关：它强调“trajectory 是对象”，与你的 multi-step tool strategy 天然一致。[44]
没解决什么：仍在 reward/judge 范式内；对“如何把 next-state directive 对齐到 latent decision”没有直接方法。[45]

RLAnything（2026）
解决什么：强调动态 forging environment/policy/reward 的闭环优化，结合 step-wise 与 outcome 信号，声称对 GUI agent/text-game/coding 等任务有效。[46]
与你最相关：表明 agentic RL 正在走向“系统级闭环 + 密集信号”。[47]
没解决什么：对你的方向而言，它更像基础设施/系统式组合；如果你只做“把 latent action 接进这个系统”，审稿人会质疑问题定义是否够“机制”。[48]

tool abstraction / schema invariance

Hammer（ICLR 2025：Robust Function-Calling via Function Masking）
解决什么：指出 function-calling 在不同 benchmark 间波动大，原因之一是对命名约定过拟合；提出通过数据与 function masking 提升对 irrelevant functions 的敏感性并减少过拟合，强调跨 benchmark 泛化。[49]
与你最相关：它直接为“schema/wording dependence 是真实问题”提供证据。[49]
没解决什么：它依然在 **surface space**（函数名/候选函数集合）上做鲁棒性；没有抽象出“decision over affordances”。[50]

Toolscaler（EMNLP Findings 2025）
解决什么：指出“每个工具一个新 token”会导致词表线性增长、长尾工具难学、协同关系稀疏；提出结构化语义 tokenization，把工具编码成离散 code 序列、相似工具共享 subtokens，并用多阶段训练利用下游协作信号，声称可在 47k 工具规模上改进。[51]
与你最相关：这是“摆脱 tool identity memorization”的强尝试之一：通过 **分解 ID** 让相似工具共享表示。[52]
没解决什么：它解决的是“工具表示与检索/生成的可扩展性”，而不是“在状态下为何选择某类工具策略”的 latent decision；本质仍在 y_t 的离散编码上优化。[53]

ToolWeaver（ICLR 2026）
解决什么：指出 retrieval-based pipeline 有 retriever 语义不足与 LLM 缺 tool knowledge 的双重问题，而 generative tool ID 又会词表爆炸；提出把工具编码成 hierarchical sequences，使词表增长对数级，并把 intrinsic semantics + extrinsic co-usage patterns 编织进 code。[54]
与你最相关：它在“工具空间变化/规模扩大”场景下确实更接近“工具之间关系/协同可迁移”。[55]
没解决什么：它仍然把“正确工具/正确序列”当作输出标记问题；没有把 next-state directive（例如工具返回提示你其实该换策略）当作对 latent policy 的更新信号。[56]

ToolTok（2026，GUI agent）
解决什么：针对 GUI agent 泛化与数据稀缺，提出把操作建模成 progressive tool usage 序列，用 learnable tool embeddings + semantic anchoring + curriculum 学习，并宣称用 <1% 数据达到强泛化。[57]
与你最相关：它说明“工具语义可学习、且语义锚定能显著提高数据效率/泛化”。[58]
没解决什么：它更像 learned tool embedding 的训练范式，尚未触及 multi-modal tool strategy 的 policy 建模（更未触及 hindsight→latent posterior）。[59]

Online-Optimized RAG（2025）
解决什么：把 tool retrieval 视为 embedding space 的分类/匹配，指出部署中 embedding misalignment 会导致错误检索；提出用最小反馈（如 task success）在线更新 item embeddings，强调 plug-and-play 与动态

工具库。 [60]
与你最相关：它从另一个角度说明“工具描述/embedding 的表层漂移会破坏系统”，也证明“环境反馈可以被用来修系统”的可行性。 [61]
没解决什么：它修的是 retriever 的几何，而不是 agent 的决策抽象；更像 tool selection pipeline 的工程补丁。 [62]

ToolFlood（2026）
解决什么：提出对检索层的攻击：注入少量“语义覆盖”工具，用 embedding 几何把 benign tools 挤出 top-k，报告在 ToolBench 上可达很高攻击成功率，并给出理论分析。 [63]
与你最相关：它是一个很硬的证据：**“检索—候选—再决策”的 pipeline，本质上把决策压缩成 embedding 邻近性**，会在规模化时暴露结构性脆弱。 [64]

Affordances Enable Partial World Modeling with LLMs（2026）
解决什么：提出并形式化“affordance 相关的部分世界模型”观点，证明实现 task-agnostic、语言条件 intent 的 agent 必然具备由 affordances 连接的 predictive partial-world model，并在桌面机器人任务中展示 affordance-aware partial models 可降低搜索分支因子。 [65]
与你最相关：这是你想要的 “decision over affordances” 在理论上的支撑点：它把“工具/动作抽象”从字符串提升到 **可预测的状态—动作子结构**。 [66]
没解决什么：它主要在机器人搜索；如何将 affordance 抽象与 LLM tool-use 的 schema 变化、next-state directive 对齐，需要你落地成可证伪的建模问题。 [67]

关键空白

现有覆盖了什么

到 2026 为止，你关心的三块（action abstraction、multi-modal policy、hindsight feedback）分别“有人在做”，但很少有人把它们合在一个可检验的 operator 里：

- **Action abstraction（$s \to z$）**：CoLA 系列给出一个清晰原型：用逆动力学从未来观测中推 latent action，并用小 codebook 把动作空间压缩，改善 RL 探索或语义多样性。 [68]
- **Multi-modal policy 建模**：连续控制社区对 flow/diffusion 的论证已相对成熟（多峰、复杂分布、覆盖候选），FlowRL 甚至明确把“capturing complex, multimodal action distributions”写进动机。 [69]
- **Hindsight / rich feedback（$s_{t+1} \to$ 学习信号）**：OpenClaw-RL/SDPO 等把 next-state directive 系统性地转成密集 token-level 监督；PRIME/ReasonFlux-PRM/RLAnything 等则在 process reward/trajectory 信号上强化 credit assignment。 [70]
- **Schema/工具空间变化**：Hammer、Toolscaler、ToolWeaver、ToolTok、Online-Optimized RAG 等，分别从 masking、层级编码、语义锚定、在线更新等角度减少对 tool ID 的硬记忆。 [71]

尚未被真正解决的部分

你要写成顶会问题的“硬缺口”在于：**反馈抽象与动作抽象脱节**。

更具体地说，当前主流 pipeline 是：

- 生成 y_t（文本/工具调用），执行得到 s_{t+1}
- 从 s_{t+1} 抽取提示或 reward
- 更新的是 $p_\theta(y_t \mid s_t)$ 或其 token decomposition

OpenClaw-RL/SDPO 就是把“反馈条件下的 token 分布”蒸馏回“无反馈的 token 分布”，从而修 y_t。 [2]

而你真正需要的是：

- 学到一个 z_t 的空间，使其对应 **决策抽象**（工具策略/子目标/参数化意图）
- 并且能从 s_{t+1} 构造 “better decision 的后验” $q(z_t \mid s_t, s_{t+1})$，再把它回传到 $p_\theta(z_t \mid s_t)$
- 最终只把 z_t 映射到 y_t 作为 realization（可由 AR、或可编译的结构化解码器负责），从而把 schema 变化隔离在 $z \to y$ 的编译层

在 2025–2026 的你给定候选里，Latent-DARM 与 STAR-LDM 给了“latent interface/latent planning”方向的形状，但它们并未把 next-state directive 显式变成 z 的后验学习目标。 [72]

看起来相关但其实不能解决你问题的点

- **“工具 tokenization / 层级 tool ID”**（Toolscaler/ToolWeaver/ToolTok）：它们解决的是“工具表示可扩展/更语义/更省数据”，但决策仍在 y 的离散输出空间做匹配；当用户意图对齐到“多条合理策略”时，它们并不提供一种对 multi-modality 友好的 policy 表达，更不提供 hindsight→latent posterior 的通道。 [73]
- **“把 diffusion 直接用作语言 backbone”**：Bitter Lesson 指出 dLLM 在 tool-calling 的符号精度上很不可靠；即使 DLLM Agent 报告效率优势，也承认 structured tool-call failures 更突出、需要更强专项训练与 masking 对齐。把 diffusion 放在 y_t 很可能会在审稿中被质疑“为何不把 diffusion 放在更适合的 latent/plan 层”。 [74]
- **“CoLA 直接迁移到 tool use”**：如果只是把 codebook action 接到 function-calling 轨迹上，最大贡献可能被审稿人解读为“压缩 action space 的工程改造”，而不是“新的 decision 学习机制”；并且 NeurIPS 2025 对 latent action 学习的理论诊断会强化这种质疑：latent 可能只是压缩码，不是语义决策。 [75]

针对你的关键判断题的结论

CoLA 路线最大局限
CoLA（及其 2026 扩展）最强的贡献是“把动作空间变小”，但它的 latent 主要通过“未来观测重构/预测”来学习，天然存在语义漂移与退化为压缩码的风险；并且它没有把 next-state directive 纳入 latent-policy 的更新机制。NeurIPS 2025 对 LAM 的分析进一步支持这种担忧：若观测变化包含外生噪声或数据生成策略偏置，latent 可能学到的是“差分主成分”而不是“可控决策因素”。 [76]

Diffusion / flow 用在 latent decision space：必要性判断
这不是“天然就该做”，但也不是“伪创新”。它在以下设定里最合理：
- **高熵、多峰的上层策略**（同意图多条合理 tool sequences；或需要在 plan 层可修订）——此时 flow/diffusion 对“多峰分布/候选覆盖”的优势有明确动机（FlowRL 的论证 + CADD 的 mode balance）。 [77]
- **你能把 $z \to y$ 做成可约束的 realization**（例如结构化解码/可编译调用），避免 diffusion 噪声直接破坏 JSON/tool schema（Bitter Lesson 给出反例；STAR-LDM 给出“diffusion 放 latent”的正例）。 [33]

它在以下设定里可能不值得：
- **低熵/近确定的工具选择**（多数现实产品的 tool routing 可能就是 top-1 或 top-2，multi-modality 不强），此时 diffusion 的采样成本与训练复杂度（pi_RL 所示的 likelihood/梯度问题）可能抵不过收益。 [78]

更重要的是：就算你不用 diffusion，你仍然可以用 **mixture policy / energy-based latent policy / posterior inference（变分后验）** 来建模多峰与后见校正；SAGE 这类“diffusion 覆盖 + energy 选优”的设计也提示 diffusion 很可能是 可选组件，不是核心贡献。 [79]

OPD / hindsight 能否与 latent action 结合：三层可行性
- 只蒸馏 output token：OpenClaw-RL/SDPO 已证明这条路能把 rich feedback 变成密集信号，但它主要修

y；对 schema invariance 的迁移性有限。 [2]
- 蒸馏 latent plan / latent action：文献几乎空白，但从 STAR-LDM（latent planning）与 Latent-DARM（latent interface）可以推断这是合理形态；你需要定义 $q(z \mid s, s_{t+1})$ 的构造方式与可证伪收益。 [80]
- 用 hindsight 反向参与 latent space discovery / codebook formation：CoLA/coverage-enhanced latent actions 强调 codebook 覆盖与学习机制，但没有把 next-state directive 作为“聚类/离散化/几何塑形”的监督；这层如果做对，会是高 insight，但风险最高（容易被质疑不稳定、评价困难）。 [75]

综合可落地性与研究价值：**第二层（latent distillation / posterior refinement）最像 2026 级别的“机制问题”**；第三层可作为加分的 ablation/扩展，而不是主线。 [81]

“tool use as policy, not pattern”的最佳 framing
在你给的备选中，最有顶会潜力的不是单点（“diffusion latent policy”或“schema-invariant”），而是能一锤定音地把两类抽象统一起来的 framing：
unified action abstraction + feedback abstraction（用 next-state directive 在 latent decision space 做后验校正，并通过 affordance/realization 分离获得 schema invariance）。
理由：它直接覆盖 $s \to z \to y$ 以及 s_{t+1} 对 z 的更新；现有工作分散覆盖其中子边（CoLA 覆盖 $s \to z$、OpenClaw/SDPO 覆盖 $s_{t+1} \to y$、ToolWeaver/Toolscaler 覆盖 y 的表示），但“闭环对齐”缺失正是你的空白。 [82]

候选研究命题

命题：后见后验驱动的潜在决策校正

一句话核心命题：**把工具使用看作 z_t 上的潜在决策，next-state directive 用来构造 $q(z_t \mid s_t, s_{t+1})$，并用后验匹配而非 token 蒸馏来更新 policy。** [81]

方法草图：
(1) 定义 latent decision z_t（连续或离散+连续残差），并学习可执行 realization $p(y_t \mid z_t, s_t)$（可 AR 或结构化编译器）；(2) 从执行后的 s_{t+1} 抽取 directive，构造“反馈条件下的 teacher”来近似后验 $q(z_t \mid s_t, s_{t+1})$；(3) 最小化 $D_{\mathrm{KL}}(q(z \mid s, s') \parallel p_\theta(z \mid s))$ 或其等价的 score-matching 形式。OPD/SDPO 的 token 蒸馏对应的是在 y 上做这件事；你把它上移到 z。 [83]

为什么比“CoLA + trick”更强：CoLA 的主收益是压缩动作空间；这里的主贡献是 **把反馈对象从 y 改为 z**，并把“决策语义”写进可辨识的后验更新算子（posterior refinement），从而天然服务 schema-invariance。 [84]

最关键实验：
在同一 tool-use 基准（可复用 BFCL-v3 与多轮 tool-calling设置）做 **schema perturbation / tool renaming / argument reshaping**：比较 token-level OPD/SDPO vs latent-posterior 更新在“工具名改写/描述替换/参数组织变化”条件下的性能退化曲线。Bitter Lesson 已经表明 tool-calling 对符号精度敏感；你的方法应展示“决策不变、realization 可适配”带来的鲁棒性优势。 [85]

最大风险点：如何构造可识别的 $q(z \mid s, s')$ 而不退化成“把 z 设成隐藏的 token 压缩”；NeurIPS 2025 对 LAM 的分析会被审稿人拿来质疑 latent 语义。你需要用消融证明：仅有 reconstruction/压缩不够，必须有 hindsight posterior 约束才能让 z 变成决策语义。 [86]

预期审稿人质疑：
“这不就是把 OPD/SDPO 换个空间？”你的回答应是：OPD/SDPO 的 teacher/student 差分在 token 空间，

无法解释 schema invariance；而你的后验是定义在决策变量上，并能在 realization 层替换编译器/工具编码仍保持策略一致。 87

命题：面向多峰工具策略的生成式 latent policy

一句话核心命题：**用 flow/diffusion 在 z_t 或 $z_{1:T}$ 上建模多峰工具策略分布，并用约束/能量筛选保证可执行性与低幻觉。** 88

方法草图：
- $p_\theta(z_{1:T} \mid s)$ 用 flow matching 或 diffusion（或“diffusion 覆盖 + energy gating”）生成多个策略候选；
- 通过 learned feasibility energy（类似 SAGE 的 latent consistency）或 tool-execution simulator 的可验证信号对候选打分，选择/重加权；
- realization 层用结构化解码生成 tool calls，避免 diffusion 直接在 JSON token 上累积噪声（吸取 dLLM tool-calling 失败经验）。 89

为什么比“在 CoLA 上加 trick”更强：CoLA 主要把 action space 变小；这里的机制点是 **多峰策略的可控覆盖**（FlowRL/CADD 的动机）以及 **可执行性筛选算子**（SAGE）。 90

最关键实验：
挑选天然多解的 tool tasks（同意图可多工具、多顺序、多参数化），对比：单峰 AR policy / 离散分类 routing / mixture policy / diffusion-flow latent policy，在“多样性-成功率-调用成本”三者权衡上的 Pareto 曲线。CADD 明确讨论 mode-coverage vs mode-seeking 可调；你的方法应展示可调性与在低熵情形自动收敛。 91

最大风险点：采样成本与训练稳定性（pi_RL 的问题提醒）；以及评价是否能公平比较（不同方法采样预算不同）。 92

预期审稿人质疑：
“为什么非得 diffusion/flow？mixture-of-experts 不行吗？”你需要在实验里明确给出对照：在相同采样/参数量下，diffusion/flow 是否提供额外可控性或更好的覆盖-精度折中。否则 diffusion 很容易被判定为“自然但非必要”。 93

命题：基于 affordance 的 schema-invariant latent decision

一句话核心命题：**把工具决策对象从 tool ID 改为 affordance（可预测的状态—动作子结构），学习 z_t 作为 affordance-conditional 的部分世界模型接口，从而实现工具空间变化下的可迁移策略。** 94

方法草图：
- 学习工具/函数的 affordance embedding（输入条件、输出类型、可验证性、失败模式等），并把工具调用视为“在 affordance 空间里选择操作”；
- policy 输出的是 affordance-level 决策 z_t，再由 tool compiler 在当前工具库中实例化成具体调用 y_t；
- next-state 信号（报错/返回）更新的是 affordance-level belief（例如“这个 affordance 在当前上下文不可行”），而不是把错写回某个具体 tool name。 95

为什么比“CoLA baseline + OPD”更强：它把“工具字符串变化导致的非平稳性”作为问题定义核心，并用 affordance 这一抽象对象把决策与 realization 解耦；ToolWeaver/Toolscaler 做的是“工具 token 更语义”，但还没把决策对象换成 affordance。 96

最关键实验：
在工具库动态变化（新增/重命名/描述替换/参数重排）下测试 zero-shot / few-shot transfer，比较：

ToolWeaver/Toolscaler 类“语义 ID”方法 vs affordance-level policy。ToolFlood 可作为压力测试：看你的 affordance 决策是否能绕开 embedding top-k 被覆盖的脆弱点。[97]

最大风险点：affordance 的可观测性与标注/自监督来源；如果 affordance 仍然靠 tool description 诱导，审稿人会认为只是“换了个 embedding”。你需要 next-state 信号作为 affordance 学习的关键监督来源。[98]

预期审稿人质疑：
“这是不是在做新 benchmark？”你要强调任务与评测可复用现有 tool-calling/agent benchmarks，只是加入 controlled schema perturbation 作为 stress test（类似 Hammer 指出的命名过拟合问题）。[99]

最终推荐

在三条命题里，我最推荐的是：**后见后验驱动的潜在决策校正（命题一）**。

为什么它最有 insight

它把 2025–2026 三条看似独立的潮流（latent action、rich feedback/OPD、schema 变化鲁棒性）收敛成一个明确、可证伪的学习算子：

- OpenClaw-RL/SDPO 证明了 next-state directive 能提供比标量 reward 更密的信息；但它们把信息注入 token。[2]
- CoLA 证明了 latent action 能改善探索与效率；但 latent 的语义对齐并不保证，且没有 feedback abstraction。[100]
- Hammer/ToolFlood 说明 surface matching 与 embedding retrieval 在 schema 变化与规模化时会脆弱。[7]

你的命题把三者链接成：**next-state 形成 z 的后验 → 用后验更新 $p(z \mid s)$ → realization 层再适配 schema**。这正是你写下的 $s \to z \to y$ 统一视角缺失的那一块。[80]

为什么它比“CoLA baseline + OPD”更像真正的问题

“CoLA + OPD”很容易被审稿人视为两个现成模块拼装：一个压 action space，一个在 token 上蒸馏纠错。你的问题如果写成 **“从 next-state directive 恢复 latent decision 后验，并以此定义 policy 更新”**，那么：

- 贡献不再是“更强训练 recipe”，而是“反馈的统计对象改变了”（从 token 序列到潜在决策），属于 operator / objective 层创新；
- 可证伪：如果你做 schema perturbation 仍然不更鲁棒，或 z 仍退化为压缩码，你的假设就失败。[101]

diffusion / flow 在这里是否必要

在推荐命题里，**diffusion/flow 应该被定位为“可选的 $p(z \mid s)$ 参数化方式”，而不是论文的唯一卖点**：

- 当工具策略确实强多峰时，用 flow/diffusion 建模 $p(z \mid s)$ 有充分动机（FlowRL 的表达力论证；CADD 的 mode balance 可控性）。[93]
- 但如果实验显示多峰性不强，或采样/训练代价过高（pi_RL 指出的 objective/likelihood 难题），你完全可以用 mixture/energy-based posterior 来实现同样的“后验校正”机制，并把 diffusion 留作扩展实验。[102]

为什么它对 tool use / function calling / agent learning 社区有意义

它把 tool use 从“输出格式正确”推进到“决策对象正确”，并且把 next-state 信号从“打分/写提示”提升为“后验推断证据”。这不仅对 function calling，也对 GUI agent、代码 agent、搜索 agent 等所有“执行后可观察反馈”的 agentic setting 成立（OpenClaw-RL 的统一叙事已明确这点）。[103]

强约束与收敛方式

你要把论文收敛成 2026 顶会问题，建议把强约束写成三条（并在实验中逐条击穿）：

1) **决策与 realization 解耦**：在一致的 latent z 下，仅替换 tool schema/名称/参数组织，性能不应显著退化。[104]
2) **反馈校正在 z 上生效**：给定同样的 next-state directive，token-level OPD/SDPO vs latent-posterior 更新，应体现出不同的跨 schema 迁移曲线。[105]
3) **多峰性可控**：在高熵状态保持覆盖（多策略候选），在低熵状态收敛到精准调用（mode seeking），并能以一个可解释的温度/能量/噪声强度参数控制（借鉴 CADD/SAGE 的机制语言）。[106]

只要这三条成立，你的论文就不再是“把 diffusion/OPD/CoLA 拼在一起”，而是一个清晰、机制驱动、可证伪的研究问题：**next-state directive 是否应该在 latent decision space 形成后验，并通过后验匹配学习出 schema-invariant 的策略表示？**[107]

---

[1] [2] [4] [37] [70] [81] [83] [87] [98] [103] [105] [107] https://arxiv.org/html/2603.10165v1
https://arxiv.org/html/2603.10165v1

[3] [8] [68] [76] [100] https://openreview.net/forum?id=cEKrGCFXPA¬eId=r2pe8zQ3sK
https://openreview.net/forum?id=cEKrGCFXPA¬eId=r2pe8zQ3sK

[5] [9] https://proceedings.mlr.press/v267/jia25e.html
https://proceedings.mlr.press/v267/jia25e.html

[6] [12] [13] [69] [77] [88] [90] [93] https://neurips.cc/virtual/2025/poster/119311
https://neurips.cc/virtual/2025/poster/119311

[7] [49] [50] [71] [99] [104] https://iclr.cc/virtual/2025/poster/27722
https://iclr.cc/virtual/2025/poster/27722

[10] [75] [82] [84] https://arxiv.org/html/2601.07516v1
https://arxiv.org/html/2601.07516v1

[11] [86] [101] https://openreview.net/pdf?id=DQMjemrVhe
https://openreview.net/pdf?id=DQMjemrVhe

[14] [15] [16] [78] [92] [102] https://arxiv.org/abs/2510.25889
https://arxiv.org/abs/2510.25889

[17] [18] https://arxiv.org/html/2503.00535v1
https://arxiv.org/html/2503.00535v1

[19] [20] [21] [79] [89] https://arxiv.org/abs/2603.02650
https://arxiv.org/abs/2603.02650

[22] [23] [24] https://openreview.net/forum?id=wN3KaUXA5X
https://openreview.net/forum?id=wN3KaUXA5X

25 26 27 80 https://arxiv.org/html/2602.20528v1
https://arxiv.org/html/2602.20528v1

28 29 30 91 106 https://arxiv.org/html/2510.01329v1
https://arxiv.org/html/2510.01329v1

31 33 74 85 https://arxiv.org/html/2601.12979v1
https://arxiv.org/html/2601.12979v1

32 https://arxiv.org/abs/2602.07451
https://arxiv.org/abs/2602.07451

34 35 36 72 https://arxiv.org/abs/2603.09184
https://arxiv.org/abs/2603.09184

38 39 40 https://arxiv.org/abs/2601.20802
https://arxiv.org/abs/2601.20802

41 42 https://arxiv.org/abs/2502.01456
https://arxiv.org/abs/2502.01456

43 44 45 https://arxiv.org/abs/2506.18896
https://arxiv.org/abs/2506.18896

46 47 48 https://arxiv.org/abs/2602.02488
https://arxiv.org/abs/2602.02488

51 52 53 73 https://aclanthology.org/2025.findings-emnlp.30.pdf
https://aclanthology.org/2025.findings-emnlp.30.pdf

54 55 56 https://arxiv.org/abs/2601.21947
https://arxiv.org/abs/2601.21947

57 58 59 https://arxiv.org/abs/2602.02548
https://arxiv.org/abs/2602.02548

60 61 62 https://arxiv.org/pdf/2509.20415
https://arxiv.org/pdf/2509.20415

63 64 97 https://arxiv.org/abs/2603.13950
https://arxiv.org/abs/2603.13950

65 66 67 94 95 96 https://arxiv.org/abs/2602.10390
https://arxiv.org/abs/2602.10390